KubeGuardian AI

Infrastructure Security Assessment Report

Security Score: 40/100

Generated by KubeGuardian AI

Findings Summary

Severity	Count
Critical	1
High	1
Medium	2
Low	1

Executive Summary

KubeGuardian AI reports a current security score of 40/100, indicating a precarious security posture. The platform's overall health is significantly compromised, with one Critical and one High-priority finding requiring immediate attention. These severe vulnerabilities present substantial risks to the integrity and availability of your environment.

Further analysis revealed two Medium and one Low-priority findings, collectively underscoring systemic weaknesses. This current state leaves your infrastructure highly susceptible to potential exploits. Urgent and comprehensive action is imperative to address these exposures, elevate your security baseline, and safeguard critical operations against imminent threats.

Detailed Findings

MISSING_TAGS

Severity: MEDIUM

Priority: P3

Resource: web

Risk: Governance gaps

Impact: Resource ownership and cost allocation become difficult.

Recommended Fix: Apply mandatory tags such as Environment, Owner, Team.

MISSING_TAGS

Severity: MEDIUM

Priority: P3

Resource: web

Risk: Governance gaps

Impact: Resource ownership and cost allocation become difficult.

Recommended Fix: Apply mandatory tags such as Environment, Owner, Team.

HARDCODED_INSTANCE_TYPE

Severity: LOW

Priority: P4

Resource: web

Risk: Reduced flexibility

Impact: Scaling and instance upgrades require code modifications.

Recommended Fix: Use Terraform variables for instance types.

OPEN_SECURITY_GROUP

Severity: CRITICAL

Priority: P1

Resource: web

Risk: Public internet exposure

Impact: Attackers can directly access workloads exposed through this security group.

Recommended Fix: Restrict ingress CIDR ranges to trusted IP addresses.

MISSING_BACKEND

Severity: HIGH

Priority: P1

Risk: Terraform state inconsistency

Impact: State corruption and infrastructure drift may occur.

Recommended Fix: Configure remote backend using S3 or GCS.

AI Remediation Report

As a Senior Platform Engineer, I've analyzed the provided findings. My assessment and recommendations are detailed below.

---

1. Root Cause Analysis

The presented issues highlight a systemic lack of mature Infrastructure as Code (IaC) governance, security best practices, and automated enforcement mechanisms across the platform. The root causes can be attributed to:

* **Absence of Standardized IaC Templates & Boilerplates:** The prevalence of `HARDCODED_INSTANCE_TYPE` and `MISSING_BACKEND` indicates that development teams might be starting new projects without standardized Terraform project structures or modules. This leads to inconsistent configurations, manual repetition of errors, and a failure to adopt collaborative and robust state management.

* **Inadequate Automated Policy Enforcement & Guardrails:** The recurring `MISSING_TAGS` and critical `OPEN_SECURITY_GROUP` findings are direct symptoms of a lack of automated policy checks within the CI/CD pipeline or via cloud-native governance tools. Relying solely on manual code reviews is insufficient to catch all infractions consistently, especially as the infrastructure scales.

* **Insufficient Security-First Mindset & Education:** The `OPEN_SECURITY_GROUP` issue points to either a critical gap in security awareness among engineers or the absence of "fail-safe" mechanisms that prevent the deployment of highly permissive network configurations. Urgent delivery pressures might also lead to shortcuts in security.

* **Weakened or Inconsistent Code Review Processes:** While not explicitly stated, the presence of these issues suggests that either code reviews are not consistently applied, or reviewers lack specific checklists/tools to identify and flag these types of misconfigurations effectively.

2. Recommended Fixes

1. **Implement Automated Policy-as-Code (PaC) Enforcement:**

* **Action:** Integrate tools like Open Policy Agent (OPA) with Conftest/Gatekeeper, AWS Config Rules, Azure Policy, or GCP Organization Policies into CI/CD pipelines and cloud environments.

* **Outcome:** Enforce mandatory tagging schemas, restrict overly permissive security group rules (`0.0.0.0/0`), and mandate remote backend configurations for all Terraform deployments.

2. **Establish & Promote Standardized Terraform Modules/Boilerplates:**

* **Action:** Develop and maintain a central repository of approved, reusable Terraform modules for common resources (e.g., compute, storage, networking). These modules should abstract away complexity, enforce best practices (e.g., variable usage for instance types, mandatory tags), and pre-configure remote backends.

* **Outcome:** Reduce configuration drift, improve consistency, accelerate development, and ensure adherence to best practices from the start.

3. **Enhance CI/CD Pipeline with Static Analysis & Security Scans:**

* **Action:** Integrate IaC security scanners (e.g., Checkov, Trivy, Kics) into the CI/CD pipeline to automatically scan Terraform code for vulnerabilities and misconfigurations (like open security groups or hardcoded values) *before* deployment.

* **Outcome:** Catch issues earlier in the development lifecycle, preventing insecure or non-compliant infrastructure from being provisioned.

4. **Security Awareness & Best Practice Training:**

* **Action:** Conduct mandatory, regular training for all engineering teams on secure cloud infrastructure practices, the principle of least privilege, network segmentation, and IaC best practices.

* **Outcome:** Elevate the overall security posture and embed a "security-first" culture within the engineering organization.

5. **Proactive Remediation & Backfill:**

* **Action:** Conduct an audit of existing infrastructure to identify and systematically remediate all resources with missing tags or open security groups. Prioritize remediation based on criticality and impact.

* **Outcome:** Address existing technical debt and security vulnerabilities.

3. Priority Order

* **P1 - CRITICAL (Immediate Action Required):**

* **Remediate `OPEN_SECURITY_GROUP` instances:** Identify and immediately restrict ingress CIDR ranges for all affected security groups to only trusted sources. This is an active security vulnerability.

* **Configure `MISSING_BACKEND` for active projects:** Implement remote backends (e.g., S3, GCS) for all in-flight or actively managed Terraform projects to prevent state corruption and enable collaborative development.

* **P2 - HIGH (Address Urgently):**

* **Implement Automated Security PaC:** Deploy and enforce policies (e.g., OPA, Cloud Policy) to block `OPEN_SECURITY_GROUP` and mandate secure network configurations in CI/CD.

* **Implement Mandatory Tagging PaC:** Deploy and enforce policies for mandatory tags (Environment, Owner, Team) for all new resource deployments.

* **Rollout Standardized IaC Boilerplates:** Introduce and require the use of standard project templates that include remote backends, variables, and common tags.

* **P3 - MEDIUM (Address Systematically):**

* **Remediate `MISSING_TAGS` on existing resources:** Audit and backfill tags for existing resources, prioritizing critical/cost-intensive assets.

* **Refactor `HARDCODED_INSTANCE_TYPE` in key modules:** Update shared modules and frequently deployed code to use variables for instance types.

* **P4 - LOW (Continuous Improvement):**

* **Integrate IaC Linting/Static Analysis:** Add tools to CI/CD pipelines to catch `HARDCODED_INSTANCE_TYPE` and similar best practice violations.

* **Conduct Ongoing Security Awareness Training:** Regular education and documentation updates for all teams.

4. Example Terraform/Kubernetes Snippets

1. Enforcing Mandatory Tags (Terraform)

```terraform

modules/common-tags/main.tf

locals {

common_tags = {

Environment = var.environment

Owner = var.owner
```

Team = var.team
ManagedBy = "Terraform" Best practice to identify IaC managed resources
}
}
resource "aws_instance" "example" {
... other instance configuration
instance_type = var.instance_type Using a variable from the next example
ami = "ami-0abcdef1234567890"
tags = merge(local.common_tags, {
Name = "${var.environment}-example-server"
Specific tags for this resource
Component = var.component_name
})
}
```
2. Using Variables for Instance Types (Terraform)
```terraform
variables.tf
variable "instance_type" {
description = "The EC2 instance type to deploy."
type = string
default = "t3.medium" Provide a sensible default
validation {
condition = contains(["t3.micro", "t3.small", "t3.medium", "t3.large"], var.instance_type)
error_message = "Invalid instance type. Please choose from allowed types."
}
}
main.tf (example resource using the variable)
resource "aws_instance" "app_server" {
ami = "ami-0abcdef1234567890"
instance_type = var.instance_type Correctly references the variable
... other configuration
}
```
3. Restricting Security Group Ingress (Terraform)
```terraform
main.tf
resource "aws_security_group" "web_app_sg" {

```
name = "web-app-sg"
description = "Security group for web application"
vpc_id = aws_vpc.main.id
Allow HTTP from specific internal CIDR for load balancer health checks
ingress {
description = "Allow HTTP from internal LB"
from_port = 80
to_port = 80
protocol = "tcp"
cidr_blocks = ["10.0.0.0/16"] Example: Internal VPC CIDR for ALB/NLB
}
Allow HTTPS from a dedicated public load balancer security group
ingress {
description = "Allow HTTPS from public ALB"
from_port = 443
to_port = 443
protocol = "tcp"
security_groups = [aws_security_group.public_alb_sg.id] Reference other SG
}
Allow SSH from a dedicated bastion/jumpbox security group
ingress {
description = "Allow SSH from bastion host"
from_port = 22
to_port = 22
protocol = "tcp"
security_groups = [aws_security_group.bastion_sg.id]
}
Standard egress rule (outbound traffic)
egress {
from_port = 0
to_port = 0
protocol = "-1"
cidr_blocks = ["0.0.0.0/0"]
}
tags = {
Name = "web-app-sg"
Environment = var.environment
}
```

}
Example of a highly restricted bastion security group
resource "aws_security_group" "bastion_sg" {
name = "bastion-sg"
description = "Security group for bastion host"
vpc_id = aws_vpc.main.id
ingress {
description = "Allow SSH from trusted office VPN IP"
from_port = 22
to_port = 22
protocol = "tcp"
cidr_blocks = ["203.0.113.0/24"] ONLY trusted office IP range
}
... other highly restricted rules
}
```
4. Configuring Remote Backend (Terraform)
```terraform
versions.tf
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~> 5.0"
}
}
MANDATORY: S3 Backend configuration for state management
backend "s3" {
bucket = "platform-terraform-state-myorg" Unique S3 bucket name
key = "project-x/environment/app-name/terraform.tfstate" Path to state file
region = "us-east-1" Backend bucket region
dynamodb_table = "platform-terraform-lock-myorg" DynamoDB table for state locking (P1)
encrypt = true Enable encryption for state file (P1)
}
/*
Example for Google Cloud Storage (GCS) backend
backend "gcs" {
bucket = "platform-terraform-state-myorg"

```
prefix = "project-x/environment/app-name"
}
*/
}
```